# Reproducibility

# Reproducibility I

Reproducibility in GNN is not an aspiration layered on top of the system; it is the operating contract that the pipeline enforces. The 0–24 processing steps are deterministic given a model specification and a target directory, and every published claim in this manuscript is traceable to a command that regenerates the underlying artifact. This section lists only commands that exist in the repository, so that a reader with a clean checkout can reproduce the pipeline, the validation gates, and this manuscript itself.

## Pipeline Smoke Run I

The fastest way to confirm a working installation is to drive the full pipeline over the discrete model family without invoking the optional LLM steps:

```
uv run python src/gnn/main.py --target-dir input/gnn_files/
```

This parses the discrete GNN files, runs visualization and rendering across the maintained backends, and writes all artifacts under the chosen output directory. The `--skip-llm` flag keeps the run hermetic and free of external API calls: the non-LLM steps all execute, the steps that would read the skipped LLM outputs record that as a warning, and the run exits 2 — the pipeline's documented warning code (0 success, 1 error, 2 warning) — rather than 0. To exercise every

registered family rather than a single one, drive the manifest through the model-family acceptance gate given below: pointing `--target-dir` at `input/gnn_files` covers that tree's 11 corpus directories. All 9 registered family target directories lie inside that tree, so a single invocation reaches every registered family.

## Validation Gates I

GNN's reproducibility guarantees rest on a small set of strict, deterministic gates that bind the manuscript's quantitative claims to recomputable ledgers. The model-family acceptance gate runs the maintained families declared in the manifest and fails on any regression:

```
uv run python scripts/run_model_family_acceptance.py \
  --manifest input/model_family_manifest.json \
  --output-dir output/model_family_acceptance --strict
```

## Validation Gates I

The semantic-fidelity gate verifies that a parse $\rightarrow$ serialize $\rightarrow$ parse round trip preserves variables, edges, dimensions, parameter shapes, equations, time semantics, and ontology mappings across the 9 model families; the cross-framework gate profiles the 7 maintained backends (PyMDP, RxInfer.jl, JAX, NumPyro, PyTorch, ActiveInference.jl, DisCoPy) — refusing any framework outside that set — and records explicit compatible and unsupported statuses rather than silently degrading. Both write their ledgers to an output directory of your choosing:

## Validation Gates I

```
uv run python scripts/run_semantic_fidelity_gate.py \
  --manifest input/model_family_manifest.json \
  --output-dir output/semantic_fidelity --strict
uv run python scripts/run_cross_framework_reliability.py \
  --manifest input/model_family_manifest.json \
  --output-dir output/cross_framework --strict
```

## Validation Gates I

Under `--strict`, each gate exits non-zero on the first mismatch, so these commands double as assertions in an automated reproduction run. Code-quality reproducibility is enforced separately through the developer command reference: `just lint` runs the Ruff linter over `src` and `scripts`, and the broader `just quality` recipe chains formatting, terminology, documentation, type, and security checks for a full pre-commit gate.

## Manuscript Reproducibility I

This manuscript is itself a reproducible artifact. Every quantitative value in the prose — the pipeline step count, the family and backend counts, the source and test inventories — is a token rather than a hard-coded literal, and the deterministic producer regenerates all of them from the tracked files at the current commit:

```
uv run python scripts/z_generate_manuscript_variables.py
```

That command recomputes the {{...}} tokens, persists them to output/data/manuscript_variables.json for audit, and hydrates the manuscript sources into output/manuscript/. The manuscript's own figures are rebuilt from the same token map:

```
uv run python -m scripts.manuscript_build_figures
```

The hydrated sources are then rendered to PDF by the docxology template's render stage. That stage lives in a separate checkout, with this repository symlinked into it at projects/active/GeneralizedNotationNotation; run from the template root:

## Manuscript Reproducibility I

```
uv run --frozen python scripts/pipeline/stage_03_render.py
  --project GeneralizedNotationNotation
```

The render needs a LaTeX installation providing the packages listed in `manuscript/preamble.md` plus `seqsplit`; the template guards `seqsplit` with `\IfFileExists`, so a missing copy degrades rather than failing the build.

## Manuscript Reproducibility I

Because the variables file is regenerated before rendering, the counts in the rendered PDF track the repository state at the commit recorded in `output/data/manuscript_variables.json` (d3c391a78): a code change that alters, for example, the test inventory (450 test files, 4808 test functions) propagates into the prose on the next regeneration without any manual editing.

# Reproducibility Contract I

# Reproducibility Contract I

- Do not cite results that cannot be regenerated or directly traced to a command in this repository.
- Keep generated outputs under `output/` and maintained manuscript source under `manuscript/`; treat everything in `output/` as disposable and regeneratable.
- Express every quantitative claim in the prose as a double-brace `{{...}}` token substituted by `scripts/z_generate_manuscript_variables.py`, never as a hard-coded number.
- Keep private data, credentials, and unpublished sensitive details out of the manuscript and out of version control.
- Record the exact verification commands — the smoke run, the acceptance and fidelity gates, and `just lint` — before marking this manuscript publication-ready.